1. Two Sum Easy

Acceptance: 56.9% | LeetCode

Given an array of integers `nums` and an integer `target`, return *indices of the two numbers such that they add up to* `target`.

You may assume that each input would have ***exactly* one solution**, and you may not use the *same* element twice.

You can return the answer in any order.

Example 1:

```
**Input:** nums = [2,7,11,15], target = 9
**Output:** [0,1]
**Explanation:** Because nums[0] + nums[1] == 9, we
return [0, 1].
```

Example 2:

```
**Input:** nums = [3,2,4], target = 6
**Output:** [1,2]
```

Example 3:

```
**Input:** nums = [3,3], target = 6
**Output:** [0,1]
```

Constraints:

- `2 <= nums.length <= 104`
- `-109 <= nums[i] <= 109`
- `-109 <= target <= 109`
- **Only one valid answer exists.**

Follow-up: Can you come up with an algorithm that is less than `O(n2)` time complexity?

Claude Opus Solution

Two Sum — Complete Breakdown

1. 💡 Intuition

The core insight is reframing the question:

> Instead of asking "*do any two numbers sum to target?*", ask "*for each number I see, does its complement already exist?*"

For any element `num`, its complement is `target - num`. If we've already seen the complement, we're done. This transforms a **search problem** into a **lookup problem**.

A hash map gives us O(1) lookups — that's the unlock.

2. 📋 Approach (Pseudocode)

```
create empty hash map: { complement → index }

for each (index, num) in nums:
    if num is in the hash map:          ← complement was stored earlier
        return [hash_map[num], index]   ← found our pair!

    store hash_map[target - num] = index ← save THIS num's complement for future
```

Key trick: We store `target - num` (the complement we *need*), not `num` itself. So when a future element equals that stored key, we know we have a match.

3. 🐍 Python Solution

```python
class Solution:
    def twoSum(self, nums: list, target: int) -> list:

        # Hash map stores: { complement_needed → index_of_who_needs_it }
        # Example: if target=9 and we see num=2, we store {7: 0}
        # meaning "index 0 needs a 7 to complete the pair"
        memo = {}

        for i, num in enumerate(nums):
```

```
            # Check: is the current number someone's "missing piece"?
            # i.e., did a previous element store 'num' as its complement?
            if num in memo:
                # memo[num] = index of the element that needed 'num'
                # i       = index of 'num' itself
                return [memo[num], i]

            # Not found yet — store what WE need for future elements to find.
            # If num=2 and target=9, store {7: i} so that when we later
            # encounter 7, we instantly know it pairs with index i.
            memo[target - num] = i

        # Problem guarantees exactly one solution, so we always return above.
        # This line is never reached for valid inputs.
        return []
```

Dry Run — `nums = [2, 7, 11, 15]`, `target = 9`

Step	i	num	memo before check	num in memo?	Action
1	0	2	`{}`	❌ No	Store `{7: 0}`
2	1	7	`{7: 0}`	✅ Yes!	Return `[0, 1]`

---

4. ⏱️ Complexity Analysis

Improved Solution (Hash Map)

	Complexity	Reasoning
Time	**O(n)**	Single pass through `nums`; each hash map lookup/insert is O(1)
Space	**O(n)**	In the worst case, we store n−1 complements before finding the answer

Brute Force (for comparison)

	Complexity	Reasoning
Time	**O(n²)**	Nested loops — for each element, scan all remaining elements
Space	**O(1)**	No extra data structures

---

5. 🔄 Alternative Approaches

① Brute Force — O(n²) time, O(1) space

```
for i in range(len(nums)):
    for j in range(i + 1, len(nums)):
        if nums[i] + nums[j] == target:
            return [i, j]
```

Simple but slow. Checks every pair. Fine for tiny inputs, fails at scale.

---

② Sort + Two Pointers — O(n log n) time, O(n) space

```
# Attach original indices before sorting (since sort destroys position info)
indexed = sorted(enumerate(nums), key=lambda x: x[1])
left, right = 0, len(indexed) - 1

while left < right:
    s = indexed[left][1] + indexed[right][1]
    if s == target:
        return [indexed[left][0], indexed[right][0]]
    elif s < target:
        left += 1   # need bigger sum → move left pointer right
    else:
        right -= 1  # need smaller sum → move right pointer left
```

Elegant but the sort costs O(n log n), and we need extra space to preserve original indices. **The hash map wins on time complexity.**

🧠 Pattern Recognition

This problem teaches the **"complement lookup"** pattern — one of the most reusable patterns in array problems:

```
seen = {}
for each element:
    if (target − element) in seen → found pair
    else: record element
```

You'll see this same pattern in: - **Two Sum II** (sorted array variant) - **Three Sum** (reduce to Two Sum) - **Subarray Sum Equals K** - **4Sum**

Recognizing this pattern instantly unlocks an entire family of problems. 🎯

Codex Solution

1. Intuition

For each number `num`, we need another number:

```
complement = target − num
```

such that:

```
num + complement = target
```

The key insight is:

> Instead of checking every pair, store numbers we have already seen in a hash map so we can check in `O(1)` time whether the complement exists.

This is a classic **hash map lookup** pattern.

---

2. Approach

We iterate through the array once.

For each number:

1. Compute the number needed to reach the target.
2. Check if that needed number has already appeared.
3. If yes, return the previous index and the current index.
4. Otherwise, store the current number and its index in the hash map.

Pseudocode

```
create empty hash map seen

for each index i and number num in nums:
    complement = target − num

    if complement exists in seen:
        return [seen[complement], i]

    seen[num] = i
```

Why this works:

- We only use numbers that appeared before the current one.
- That prevents using the same element twice.
- Since the problem guarantees exactly one solution, we will always find an answer.

---

3. Python Solution

```python
class Solution:
    def twoSum(self, nums: list[int], target: int) -> list[int]:
        # Dictionary to store numbers we have already seen.
```

```
        #
        # Key:   number from nums
        # Value: index of that number
        seen = {}

        # Iterate through the array with both index and value.
        for i, num in enumerate(nums):
            # The value we need to pair with nums[i]
            # so that their sum equals target.
            complement = target - num

            # If the complement was already seen earlier,
            # then we found the two indices.
            if complement in seen:
                return [seen[complement], i]

            # Otherwise, store the current number and its index.
            #
            # We store after checking so that we do not accidentally
            # use the same element twice.
            seen[num] = i

        # The problem guarantees exactly one solution,
        # so this line should never be reached.
        return []
```

---

4. Complexity Analysis

Let `n` be the length of `nums`.

Time Complexity: `O(n)`

We loop through the array once.

Each dictionary lookup and insertion is `O(1)` on average.

Therefore:

```
O(n)
```

Space Complexity: `O(n)`

In the worst case, we may store almost every number in the hash map before finding the answer.

Therefore:

```
O(n)
```

---

5. Alternative Approaches

Brute Force

Check every pair of numbers.

```
class Solution:
    def twoSum(self, nums: list[int], target: int) -> list[int]:
        for i in range(len(nums)):
            for j in range(i + 1, len(nums)):
                if nums[i] + nums[j] == target:
                    return [i, j]
```

Complexity

```
Time:  O(n^2)
Space: O(1)
```

This is simple but too slow for large inputs.

Sorting + Two Pointers

Another common approach is:

1. Store each number with its original index.
2. Sort by number.
3. Use two pointers to find the target sum.

However, sorting takes `O(n log n)` time, so it is slower than the hash map solution.

Complexity

```
Time:  O(n log n)
Space: O(n)
```

This approach is useful in some variations, but for this problem, the hash map method is optimal.